



NutriSmart Mobile Application

Academic Technical Report

Team: NutriSmart Development Team

Rana Ben Fraj
Koussay Ghanmi

Academic Year 2024 – 2025

Abstract

NutriSmart is an Android mobile application developed with Jetpack Compose that addresses the everyday challenge of meal planning, dietary management, and food waste reduction. Built on the Model-View-ViewModel (MVVM) architectural pattern, the application integrates a local Room database for persistent storage, a Retrofit-based AI service for intelligent recipe generation, and a navigation graph that protects users from runtime crashes. This report provides a comprehensive technical account of the application's architecture, data flow, core functionalities, AI integration strategy, database design, user interface decisions, and the challenges encountered during development. The document is intended for an academic audience and follows the structure typically expected of a final-year software engineering project report.

Keywords: Android, Jetpack Compose, MVVM, Room Database, AI Recipe Generation, Meal Planning, Kotlin, Retrofit.

Contents

Abstract	i
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Target Users	2
1.5 Report Structure	2
2 Functional Analysis	3
2.1 Overview	3
2.2 Primary Functions	3
2.2.1 User Authentication and Onboarding	3
2.2.2 Recipe Discovery	3
2.2.3 Recipe Detail View	4
2.2.4 Favourites Management	4
2.2.5 Daily Ideas Generation	4
2.2.6 Weekly Meal Planning	4
2.2.7 Leftover Recipe Finder	4
2.2.8 Shopping List	4
2.2.9 User Profile	4
2.3 Secondary Functions	4
2.4 Functional Requirements Summary	5
3 Software Architecture	6
3.1 Architectural Pattern: MVVM	6
3.2 Layer Responsibilities	6
3.2.1 Presentation Layer	6
3.2.2 Domain Layer	7
3.2.3 Data Layer	7
3.3 Component Communication	7
3.4 Dependency Injection	8

4	Technologies Used	9
4.1	Programming Language	9
4.2	Android SDK and Jetpack Libraries	9
4.3	Networking	9
4.4	Serialisation	10
4.5	UI Design	10
4.6	Build System	10
5	UML Diagrams	11
5.1	Use Case Diagram	11
5.2	Class Diagram	12
6	Database Design	13
6.1	Overview	13
6.2	Entity–Relationship Diagram	13
6.3	Entity Descriptions	14
6.4	DAO Architecture	14
6.5	Database Initialisation	15
7	User Interface Design	16
7.1	Design Philosophy	16
7.2	Navigation Flow	16
7.3	Screen Showcase	17
7.3.1	Sign-In Screen	17
7.3.2	Home Screen	18
7.3.3	Daily Ideas Screen	19
7.3.4	Recipe Detail Screen	20
7.3.5	Weekly Planner Screen	21
7.3.6	Shopping List Screen	22
7.3.7	Profile Screen	23
8	Core Features	24
8.1	Authentication and Session Management	24
8.2	Recipe Management	24
8.3	Favourites Toggle	24
8.4	Leftover Recipe Matching	24
9	Artificial Intelligence Integration	26
9.1	AI Approach	26
9.2	AI Workflow	26
9.3	Fallback Mechanism	26

9.4	Benefits of AI Integration	27
10	Testing	28
10.1	Testing Strategy	28
10.2	Manual Testing Conducted	28
10.3	Unit Testing Recommendations	28
10.4	UI Testing Recommendations	29
11	Performance and Security	30
11.1	Performance	30
11.2	Security	30
12	Challenges and Solutions	31
12.1	BuildConfig Field Resolution	31
12.2	Navigation Argument Safety	31
12.3	AI Response Parsing Robustness	31
12.4	Immediate UI Feedback for Async Operations	31
12.5	Deprecation Warnings	32
13	Conclusion	33
13.1	Summary	33
13.2	Lessons Learned	33
13.3	Future Work	34

List of Figures

3.1	MVVM Architecture – Data Flow Overview	7
5.1	NutriSmart – Use Case Diagram	11
5.2	NutriSmart – Class Diagram (Domain and Data Layers)	12
6.1	Entity–Relationship Diagram – NutriSmart Local Database	13
7.1	Authentication Screen (EnhancedAuthScreen)	17
7.2	Home Screen – Recipe Catalogue	18
7.3	Daily Ideas Screen – AI-Generated Suggestions	19
7.4	Recipe Detail Screen (RecipeDetailsScreen)	20
7.5	Weekly Planner Screen (WeeklyPlannerScreen)	21
7.6	Shopping List Screen (ShoppingListScreen)	22

7.7 Profile Screen (<code>ProfileScreen</code>)	23
---	----

List of Tables

2.1 Functional Requirements	5
4.1 Core Android and Jetpack Libraries	9
6.1 Room Database Entities	14

Chapter 1

Introduction

1.1 Background and Context

Modern lifestyles increasingly demand efficient approaches to nutrition management. Many individuals struggle to plan balanced meals, avoid food waste, and adhere to dietary constraints or budgets simultaneously. While numerous recipe applications exist on the market, most focus solely on discovery rather than offering an integrated experience that combines planning, AI-driven suggestion, leftover management, and persistent personalisation.

NutriSmart was conceived to fill this gap: a single, cohesive Android application that acts as a personal nutrition assistant, covering everything from browsing curated recipes to generating a complete weekly meal plan tailored to the user's dietary preferences and time constraints.

1.2 Problem Statement

The following pain points motivated the development of NutriSmart:

- **Decision fatigue:** Users must decide what to cook every day, often defaulting to repetitive or unhealthy meals.
- **Ingredient waste:** Leftover ingredients in the refrigerator are frequently discarded because users do not know what to cook with them.
- **Budget management:** Without planning, food expenditure tends to exceed personal budgets.
- **Dietary adherence:** People with specific dietary regimes (vegetarian, low-carb, etc.) find it difficult to consistently discover suitable recipes.

1.3 Objectives

The primary objectives of the NutriSmart project are:

1. Provide an intuitive platform for browsing and discovering recipes filtered by diet, preparation time, and budget.
2. Enable daily and weekly meal planning with one-tap AI-generated suggestions.
3. Allow users to enter available ingredients and receive recipe recommendations that minimise waste.
4. Persist user preferences, favourites, and meal plans locally for offline access.
5. Integrate an external AI service (Cerebras API) for generative recipe creation.

1.4 Target Users

NutriSmart targets a broad audience united by a desire to eat better with less effort:

- **Busy professionals** who need quick, healthy meal ideas during the working week.
- **Families** looking to plan diverse weekly menus without repetition.
- **Budget-conscious individuals** seeking to make the most of every ingredient.
- **Health-focused users** who follow specific dietary regimes and require tailored suggestions.

1.5 Report Structure

This report is organised as follows. Chapter 2 presents a functional analysis of the application. Chapter 3 describes the software architecture. Chapter 4 surveys the technologies employed. Chapter 5 provides UML modelling artefacts. Chapter 6 details the database design. Chapter 7 discusses the user interface. Chapters 8 through 11 cover core features, AI integration, data flow, and testing respectively. Chapters 12 and 13 address performance, security, and encountered challenges. The report concludes with Chapter 14.

Chapter 2

Functional Analysis

2.1 Overview

A functional analysis identifies and structures the capabilities that the system must provide to meet user and stakeholder expectations. The following analysis categorises NutriSmart’s features into primary and secondary functions and describes both user-facing behaviour and the underlying system logic.

2.2 Primary Functions

2.2.1 User Authentication and Onboarding

Upon first launch, the user is directed to the authentication flow managed by `EnhancedAuthScreen`. New users complete a registration form and are then guided through `UserHabitsFormScreen`, which collects dietary preferences, maximum preparation time, and budget range. Returning users are authenticated against locally stored credentials.

Key actions:

- Sign-up with email and password validation.
- Secure session persistence via `SharedPreferences` (`logged_in_email`).
- Habit collection: diet category, budget, and time constraints.

2.2.2 Recipe Discovery

The home screen aggregates available recipes and presents them in a scrollable card-based layout. Each recipe card displays a thumbnail, title, preparation time, and diet tags.

2.2.3 Recipe Detail View

Navigating to a recipe opens `RecipeDetailsScreen`, which displays full nutritional information, ingredient list, and preparation steps. From this screen the user can add the recipe to their favourites or their meal plan.

2.2.4 Favourites Management

Users can bookmark recipes using a heart icon on any recipe card. The favourite state is persisted via `FavoriteDao` in the Room database and reflected immediately in the UI via a `StateFlow`.

2.2.5 Daily Ideas Generation

The Daily Ideas feature uses the AI engine to generate a personalised set of meal suggestions for the current day, considering the user's dietary profile. Results are displayed in `DailyIdeasScreen`.

2.2.6 Weekly Meal Planning

`WeeklyPlannerScreen` allows users to view and edit a seven-day meal plan. AI-generated suggestions can be applied with a single action, or users may manually assign recipes to individual days and meal slots (breakfast, lunch, dinner).

2.2.7 Leftover Recipe Finder

Users enter available ingredients in `LeftoverRecipesScreen`. The system scores recipes based on ingredient overlap and returns the best matches, reducing food waste.

2.2.8 Shopping List

A consolidated shopping list is generated from the active weekly plan. `ShoppingListScreen` presents the required ingredients, quantities, and allows users to tick off items.

2.2.9 User Profile

`ProfileScreen` displays account information, dietary preferences, and application statistics. Users can update their habits here at any time.

2.3 Secondary Functions

- Error fallback screens with human-readable messages when navigation arguments are missing.
- Offline capability: all core data is stored locally via Room.

- Centralised error handling in `AppViewModel.handleError()` for consistent logging and user feedback.

2.4 Functional Requirements Summary

Table 2.1. Functional Requirements

#	Requirement	Description
FR1	Authentication	Users must be able to register, log in, and maintain a persistent session.
FR2	Recipe Browsing	The system shall display a catalogue of recipes filterable by diet and time.
FR3	Recipe Details	Detailed nutritional and instructional content must be available per recipe.
FR4	Favourites	Users may save and remove favourite recipes, persisted across sessions.
FR5	Daily Ideas	AI-generated daily meal suggestions based on user profile.
FR6	Weekly Planner	Users can build and edit a 7-day meal plan.
FR7	Leftover Finder	Ingredient-based recipe matching to reduce waste.
FR8	Shopping List	Auto-generated ingredient list derived from the weekly plan.
FR9	Profile Management	Users can view and update their dietary preferences and account details.

Chapter 3

Software Architecture

3.1 Architectural Pattern: MVVM

NutriSmart adopts the **Model-View-ViewModel (MVVM)** pattern, the recommended architecture for modern Android development. MVVM enforces a clear separation of concerns across three layers:

- **Model** – represents the data and business logic (domain entities, repositories, DAOs, AI services).
- **View** – the UI layer, built entirely with Jetpack Compose composables. Views observe `ViewModel` state and emit events upward.
- **ViewModel** – bridges the View and Model. It holds UI state as `StateFlow` or `mutableStateOf`, applies business logic, and delegates data operations to repositories.

This design ensures that the UI is purely reactive: composables rebuild whenever observed state changes, without directly touching data sources.

3.2 Layer Responsibilities

3.2.1 Presentation Layer

All screens are stateless Jetpack Compose functions. They receive data and callbacks as parameters and observe `ViewModel` state via `collectAsState()` or direct reads of `mutableStateOf` holders.

Navigation is centralised in `SafeNutriSmartNavGraph.kt`, which declares a `NavHost` and guards every route argument. For instance, the `RecipeDetails` route validates `recipeId` before allowing navigation; a missing or blank ID triggers `ErrorFallbackScreen` instead of a crash.

3.2.2 Domain Layer

The domain layer is framework-agnostic Kotlin. It defines:

- **Models:** Recipe, User, DayMealPlan, WeeklyMealPlan.
- **Repository interfaces:** contracts that data-layer implementations must fulfil.
- **AIEngine:** encapsulates local recipe filtering, scoring, and suggestion logic. It also delegates to `CerebrasRecipeGenerator` for cloud-based generation.

3.2.3 Data Layer

This layer provides concrete implementations of domain repository interfaces:

- `RecipeRepositoryImpl`, `FavoriteRepositoryImpl`, `MealPlanRepositoryImpl` — each wrapping the corresponding Room DAO.
- `CerebrasApiService` — a Retrofit service interface for AI recipe generation.
- Seed data utilities that pre-populate the database on first run.

3.3 Component Communication

Figure 3.1 illustrates the flow of data across layers.

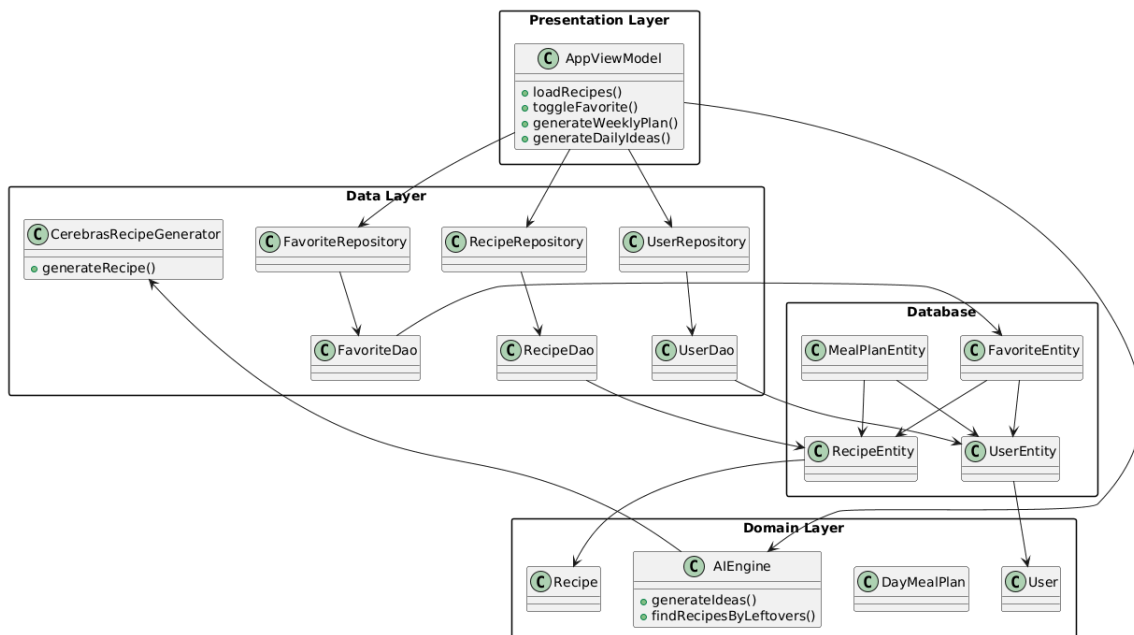


Figure 3.1. MVVM Architecture – Data Flow Overview

3.4 Dependency Injection

Dependencies are assembled in `AppContainer.kt` and injected into `AppViewModel` via a custom `ViewModelFactory`. This approach is suitable for the current project scale. A future migration to Hilt would improve testability and reduce boilerplate.

Chapter 4

Technologies Used

4.1 Programming Language

- **Kotlin** — the primary language. Its conciseness, null-safety, and first-class coroutine support make it ideal for Android development.

4.2 Android SDK and Jetpack Libraries

Table 4.1. Core Android and Jetpack Libraries

Library / Component	Role in NutriSmart
Jetpack Compose	Declarative UI toolkit replacing XML layouts. All screens are composable functions.
ViewModel	Holds UI state and survives configuration changes (e.g., screen rotation).
StateFlow / mutableStateOf	Reactive state holders observed by composables for automatic recomposition.
Room	Local SQLite abstraction providing compile-time SQL verification via DAOs.
Navigation (NavHost)	Type-safe navigation graph with argument validation in <code>SafeNavGraph.kt</code> .
Coroutines	Structured concurrency for all asynchronous operations (<code>viewModelScope</code>).
SharedPreferences	Lightweight key-value storage for session management (logged-in email).

4.3 Networking

- **Retrofit 2** — HTTP client used to call the Cerebras AI API. Interfaces are declared in `CerebrasApiService.kt`.

- **OkHttp 3** — Underlying HTTP engine; provides interceptors for logging and header injection.

4.4 Serialisation

- **Kotlinx.serialization** — Parses JSON responses from the Cerebras API into `AiRecipeDto` data classes. This library integrates with Kotlin's type system to provide safe deserialization.

4.5 UI Design

- **Material 3 (Material You)** — Provides the colour scheme, typography, and component library used throughout the application, applied via `NutriSmartTheme`.

4.6 Build System

- **Gradle (Kotlin DSL)** — Manages dependencies, configures annotation processors (kapt/ksp for Room), injects `BuildConfig` fields (`CEREBRAS_API_KEY`, `CEREBRAS_BASE_URL`), and assembles the APK.

Chapter 5

UML Diagrams

5.1 Use Case Diagram

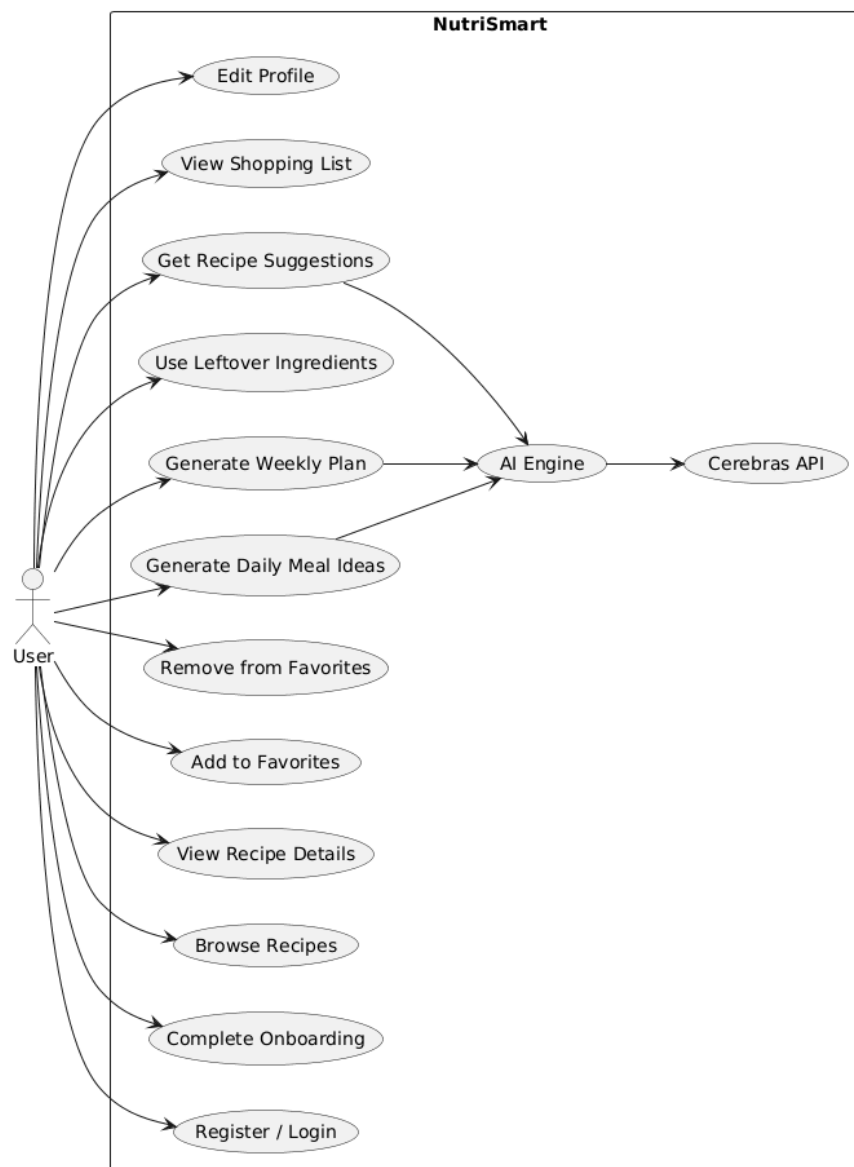


Figure 5.1. NutriSmart – Use Case Diagram

5.2 Class Diagram

Figure 5.2 presents a simplified class diagram highlighting the major domain entities and their relationships.

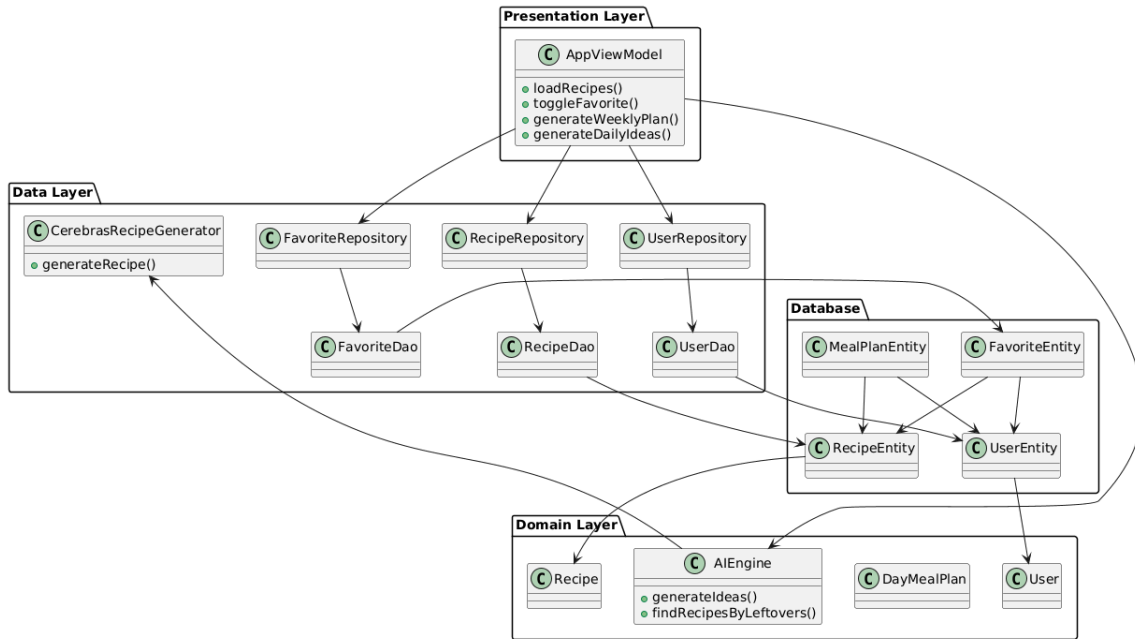


Figure 5.2. NutriSmart – Class Diagram (Domain and Data Layers)

Principal classes include `Recipe`, `User`, `DayMealPlan`, `WeeklyMealPlan`, `AppViewModel`, `AIEngine`, `CerebrasRecipeGenerator`, and all DAO interfaces.

When the user requests Daily Ideas or Weekly Plan generation, the following sequence is executed:

1. The composable calls `AppViewModel.generateDailyIdeas()` (or `generateWeeklyPlan()`).
2. The ViewModel invokes `AIEngine`, which builds a structured prompt.
3. If remote generation is enabled, `CerebrasRecipeGenerator` sends the prompt to the Cerebras API via Retrofit.
4. The JSON response is deserialized into `AiRecipeDto` and mapped to domain `Recipe` objects.
5. The ViewModel updates the relevant `StateFlow`, triggering UI recomposition.

Chapter 6

Database Design

6.1 Overview

NutriSmart uses a **Room database** (`NutriSmartDatabase.kt`) as its local persistence layer. Room provides an abstraction over SQLite with compile-time SQL verification, migration support, and seamless coroutine integration.

6.2 Entity–Relationship Diagram

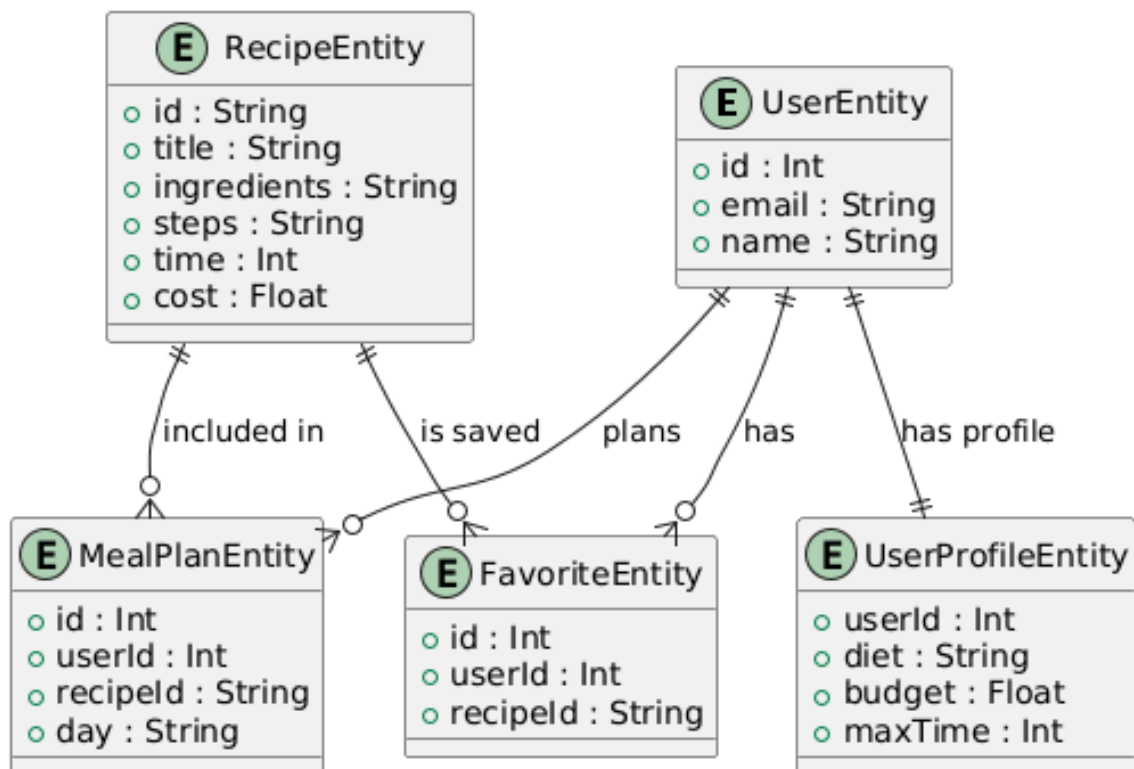


Figure 6.1. Entity–Relationship Diagram – NutriSmart Local Database

6.3 Entity Descriptions

Table 6.1. Room Database Entities

Entity	Description
UserEntity	Stores user credentials, dietary category, budget, and maximum preparation time.
RecipeEntity	Represents a recipe with fields for title, ingredients, steps, nutritional data, diet tags, and preparation time.
FavoriteEntity	Join table linking a user to their bookmarked recipes (userId, recipeId).
MealPlanEntity	Represents a single assigned meal (recipe reference, day of week, meal slot).
UserProfileEntity	Extended profile data including avatar path and preference notes.
WeeklyMealPlanEntity	Groups MealPlanEntity records into a named weekly plan.
ShoppingListEntity	Stores consolidated ingredients derived from the active weekly plan.
LeftoverInputEntity	Records ingredient names entered by the user for the leftover finder.
LeftoverRecipeResultEntity	Stores the results of leftover ingredient matching.

6.4 DAO Architecture

Each entity has a corresponding Data Access Object (DAO) that defines the SQL operations permitted:

- UserDao — insert, query by email, update.
- RecipeDao — insert all, query all, query by ID, query by diet category.
- FavoriteDao — insert, delete, query by user ID.
- MealPlanDao — insert, delete, query by week.
- Additional DAOs for shopping list, profile, and leftover entities.

All DAO methods are `suspend` functions executed within `viewModelScope`, ensuring database operations never block the main thread.

6.5 Database Initialisation

On first launch, `NutriSmartDatabase` is instantiated via a singleton builder and seeded with a curated set of recipes from the project's seed data utilities. The `fallbackToDestructiveMigration()` strategy currently handles schema version upgrades — a candidate for improvement via proper migration scripts in future releases.

Chapter 7

User Interface Design

7.1 Design Philosophy

NutriSmart's UI is built entirely with **Jetpack Compose** and styled using the **Material 3** design system. The interface prioritises clarity and ease of navigation: large recipe cards, consistent iconography, and an accessible colour palette centred on greens to reinforce the healthy-eating theme.

7.2 Navigation Flow

The application follows a hub-and-spoke navigation model:

1. **Auth** (`Screen.Auth`) — entry point for unauthenticated users.
2. **Onboarding** (`UserHabitsFormScreen`) — collected once, on first registration.
3. **Home** (`Screen.Home`) — central hub presenting recipe catalogue and quick actions.
4. **Spoke screens**: Daily Ideas, Leftover Finder, Weekly Planner, Shopping List, Profile.
5. **Detail**: Recipe Details, accessible from any recipe card across all screens.

7.3 Screen Showcase

7.3.1 Sign-In Screen

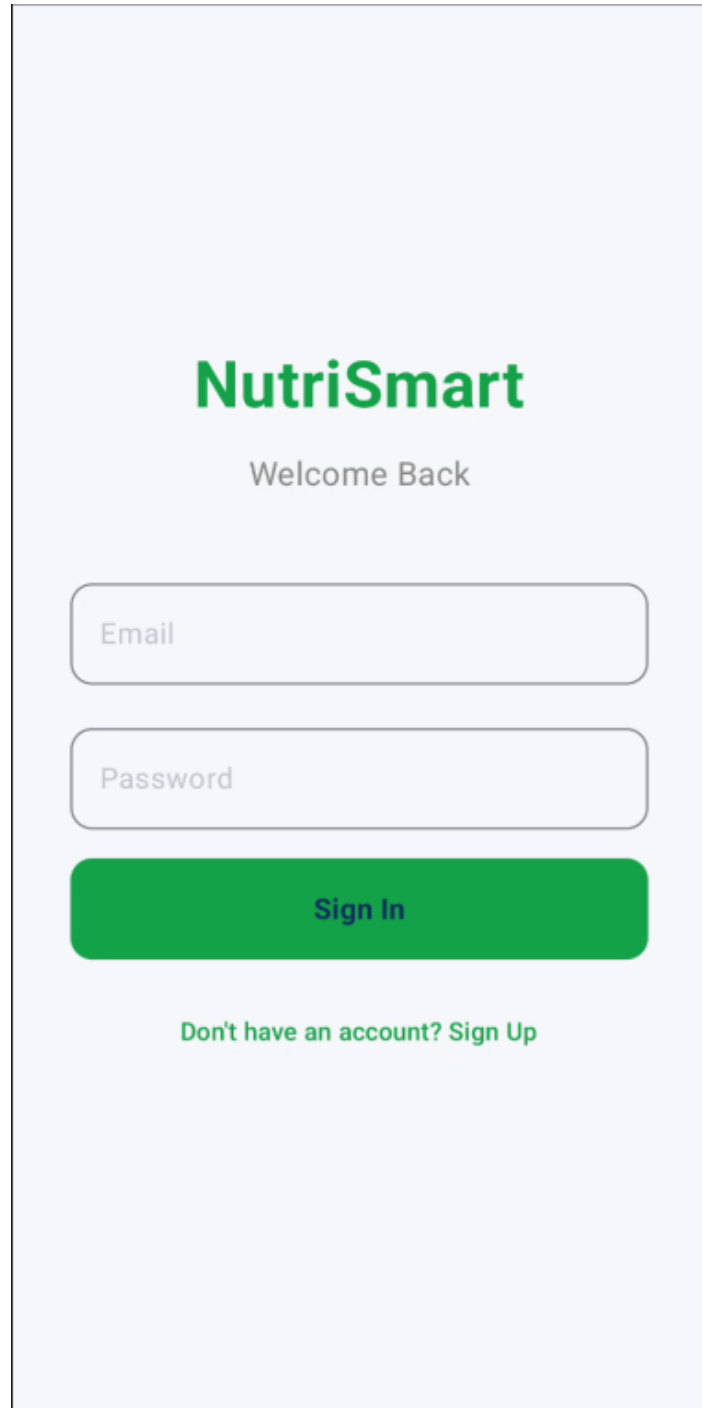


Figure 7.1. Authentication Screen (EnhancedAuthScreen)

The sign-in screen presents email and password fields with Material 3 text fields and a prominent call-to-action button. A toggle switches between login and registration

modes.

7.3.2 Home Screen

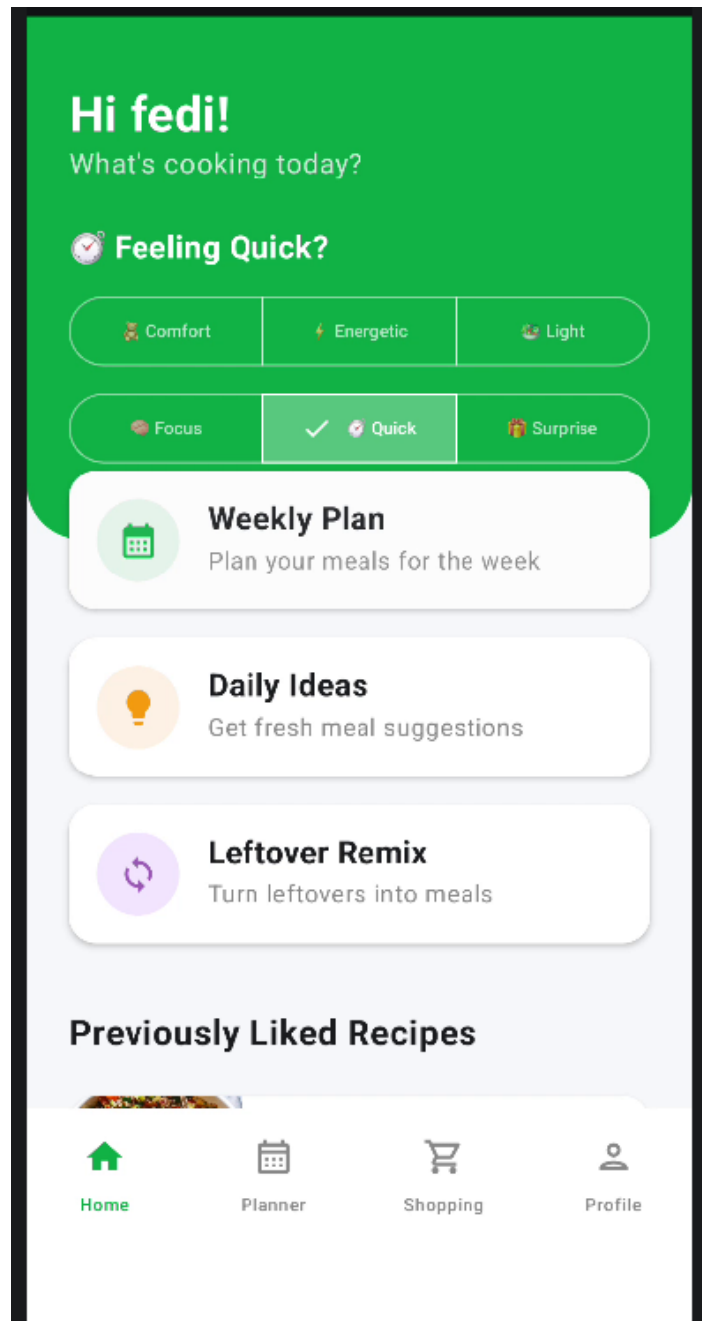


Figure 7.2. Home Screen – Recipe Catalogue

The home screen greets the user by name and presents a scrollable grid of recipe cards. Each card includes a food image, recipe title, preparation time, and a heart icon for quick favouriting.

7.3.3 Daily Ideas Screen

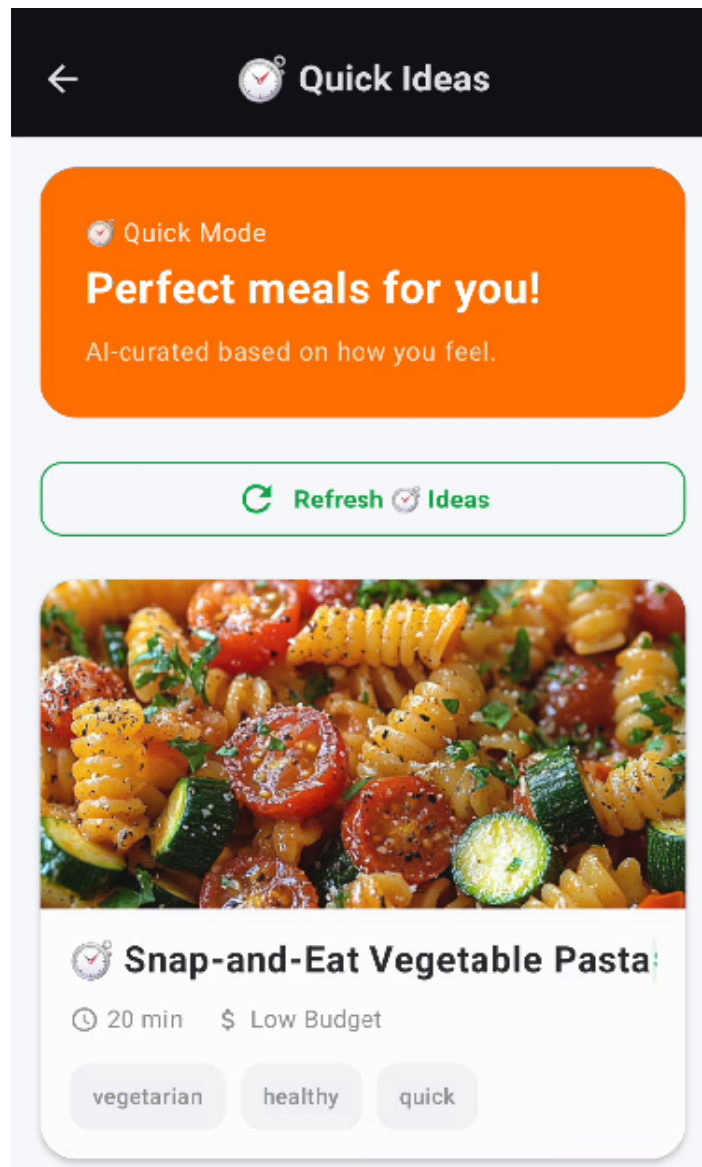


Figure 7.3. Daily Ideas Screen – AI-Generated Suggestions

This screen displays three to five AI-generated meal suggestions for breakfast, lunch, and dinner. A refresh button re-triggers the AI engine to produce a fresh set of ideas.

7.3.4 Recipe Detail Screen

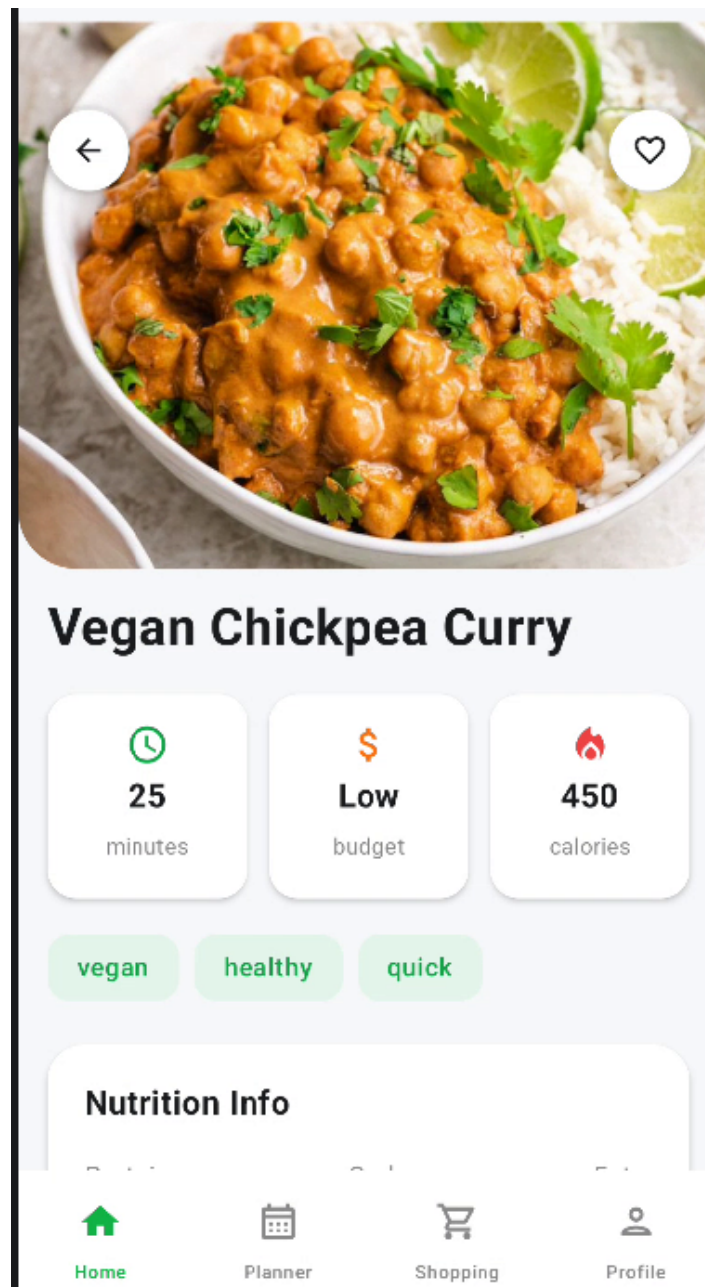


Figure 7.4. Recipe Detail Screen (RecipeDetailsScreen)

The detail view provides the full recipe: an image header, nutritional summary, ingredient list, and step-by-step preparation instructions. Action buttons allow favouriting and adding the recipe to the meal plan.

7.3.5 Weekly Planner Screen

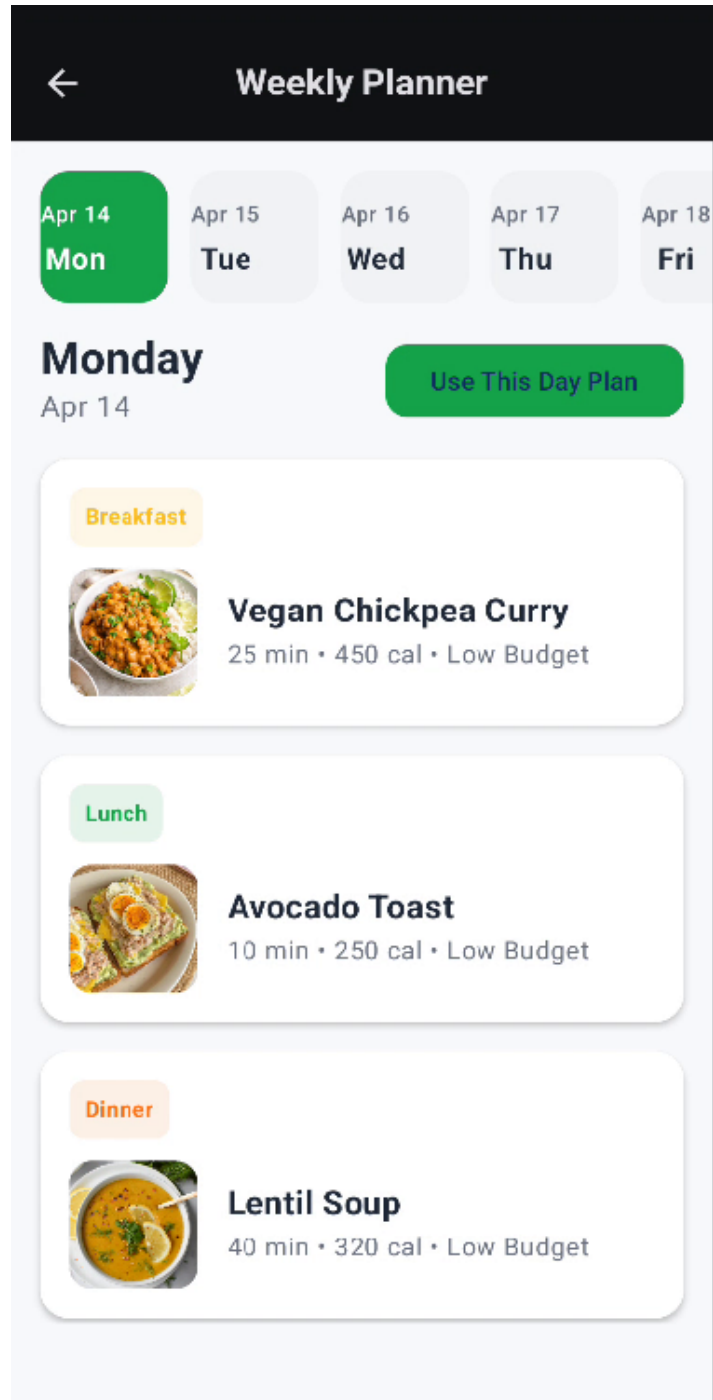


Figure 7.5. Weekly Planner Screen (WeeklyPlannerScreen)

The planner presents a seven-column day grid with meal slot rows (breakfast, lunch, dinner). Users can tap any slot to assign a recipe, or trigger AI auto-fill to populate the entire week.

7.3.6 Shopping List Screen

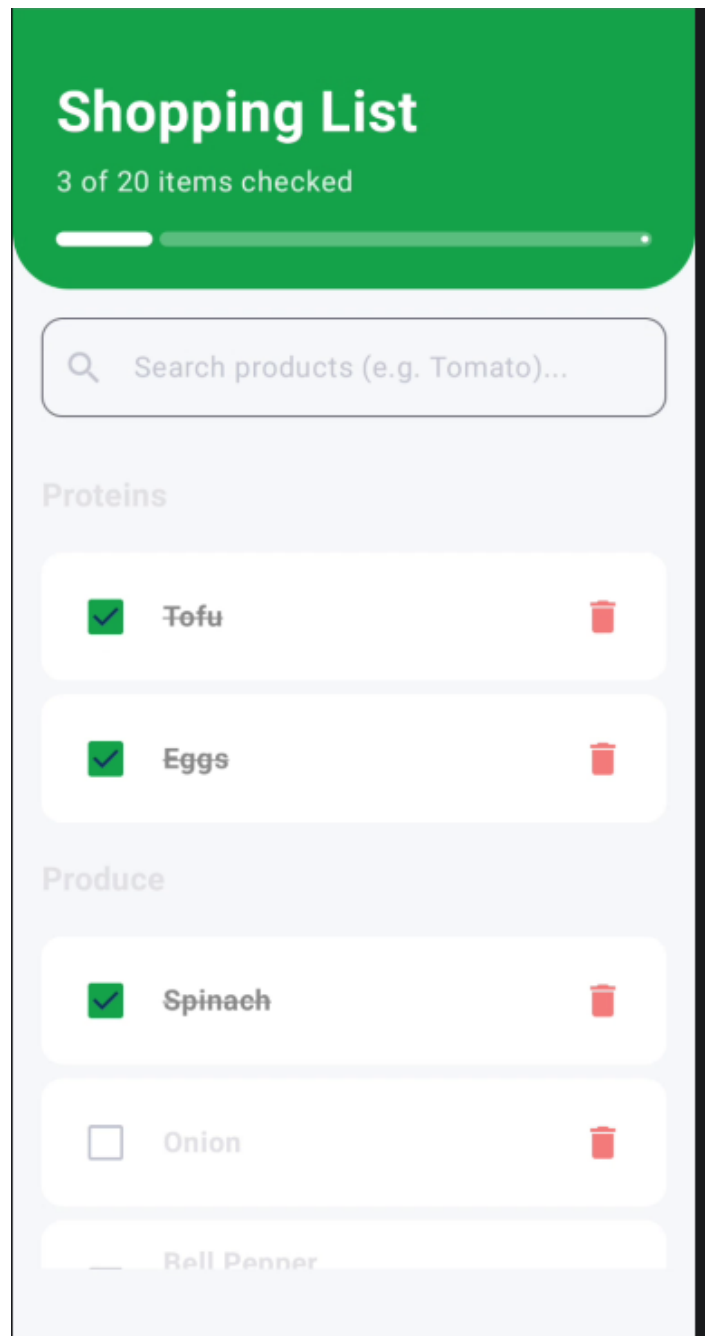


Figure 7.6. Shopping List Screen (`ShoppingListScreen`)

Ingredients from the active weekly plan are aggregated and displayed as a checklist. Tapping an item marks it as purchased. The list can be cleared or regenerated at any time.

7.3.7 Profile Screen

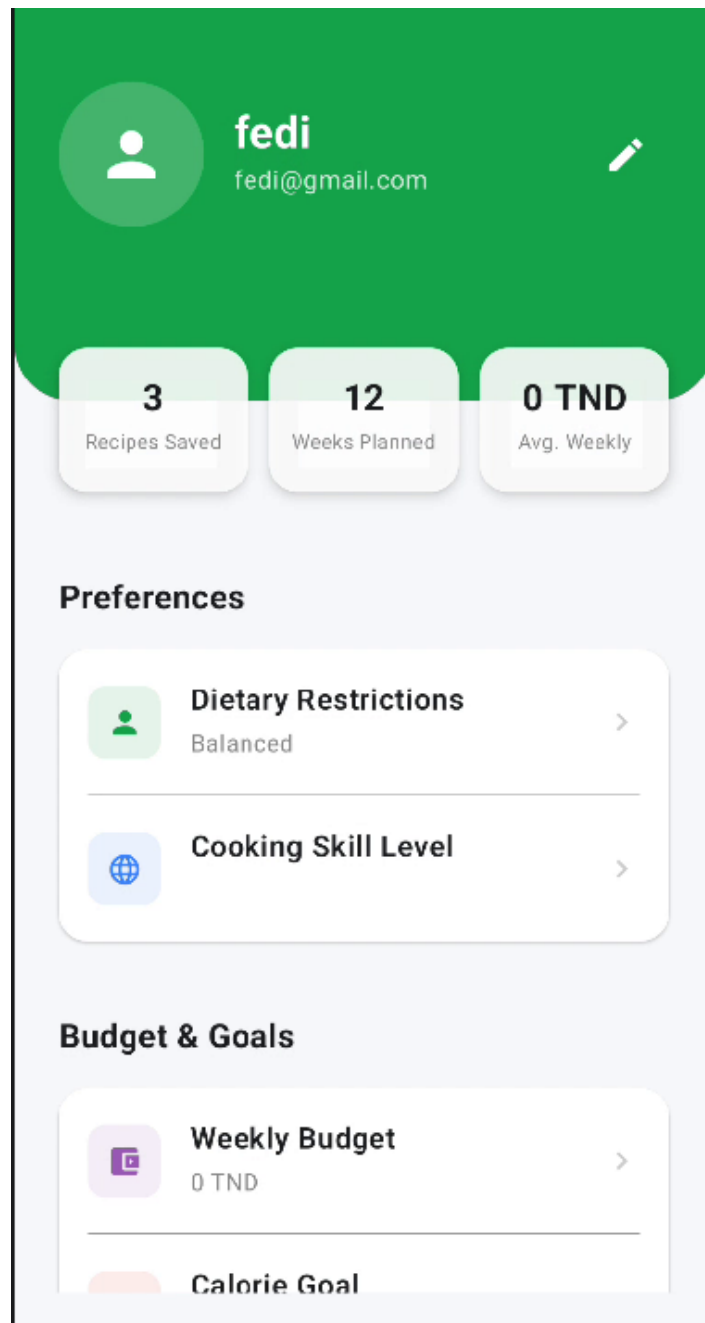


Figure 7.7. Profile Screen (ProfileScreen)

The profile screen shows the user's account information, dietary preferences, budget range, and application usage statistics. Inline editing allows preferences to be updated without leaving the screen.

Chapter 8

Core Features

8.1 Authentication and Session Management

`AppViewModel.signIn()` and `signUp()` manage the authentication lifecycle. Credentials are validated against `UserDao`, and upon successful login the user's email is persisted in `SharedPreferences`:

On subsequent launches, the `ViewModel` reads this key during `loadUser()` and bypasses the login screen if a valid email is found.

8.2 Recipe Management

Recipe data is seeded into Room on first run via seed utilities. `RecipeRepositoryImpl` wraps `RecipeDao` to provide:

- `getAllRecipes()` — retrieves the full catalogue as a `Flow<List<RecipeEntity>>`.
- `getRecipeById(id)` — used by `RecipeDetailsViewModel` for the detail screen.
- `getFavoriteRecipes(userId)` — cross-references `FavoriteEntity` records.

8.3 Favourites Toggle

The favourite toggle is one of the most frequently called operations in the application. `AppViewModel.toggleFavorite(recipe)` applies the following guard pattern: This approach avoids reloading the entire list from the database and delivers an instantaneous UI response.

8.4 Leftover Recipe Matching

`AIEngine.findRecipesByLeftovers(ingredients)` implements a scoring algorithm:

1. For each candidate recipe, count how many of its required ingredients appear in

the user's input list (case-insensitive contains match).

2. Compute a coverage ratio: `matchedCount / totalIngredients`.
3. Return recipes sorted by coverage ratio in descending order.

This deterministic fallback ensures the feature works offline when the AI API is unavailable.

Chapter 9

Artificial Intelligence Integration

NutriSmart integrates Artificial Intelligence to enhance user experience by generating personalized meal suggestions and improving decision-making.

9.1 AI Approach

The system uses a hybrid approach combining both local logic and external AI services:

- **Local Engine:** Provides basic recommendations by filtering recipes based on user preferences such as diet, budget, and preparation time.
- **External AI Service:** Generates new recipes dynamically using a remote API, allowing more diverse and personalized suggestions.

This combination ensures that the application remains functional even without an internet connection while benefiting from advanced AI capabilities when available.

9.2 AI Workflow

The process of generating AI-based recipes follows these steps:

1. The user selects a feature (daily ideas, weekly planner, or leftovers).
2. The system collects user preferences (diet, time, budget).
3. A request is sent to the AI service.
4. The system processes the response and converts it into recipe data.
5. The results are displayed in the user interface.

9.3 Fallback Mechanism

To ensure reliability, the system includes a fallback mechanism:

- If the AI service is unavailable, the application switches to local recommendations.
- Users always receive suggestions, even without network connectivity.

9.4 Benefits of AI Integration

The integration of AI provides several advantages:

- Personalized meal recommendations
- Reduced decision-making effort
- Better use of available ingredients
- Enhanced overall user experience

Chapter 10

Testing

10.1 Testing Strategy

Given the scope of this academic project, testing efforts focused on manual integration testing and code-level guard clauses rather than automated test suites. However, the architecture is deliberately structured to support comprehensive testing in future iterations.

10.2 Manual Testing Conducted

- **Authentication flow:** verified sign-up, login, session persistence across app restarts, and logout.
- **Navigation safety:** confirmed that navigating to `RecipeDetails` with a missing or blank `recipeId` triggers the error fallback screen.
- **Favourite toggle:** tested rapid successive taps to verify the optimistic UI update matches the database state.
- **AI generation:** exercised Daily Ideas and Weekly Planner with various dietary profiles; validated JSON parsing with well-formed and malformed responses.
- **Leftover matching:** entered various combinations of ingredients and verified the returned recipes were sorted by coverage score.
- **Build verification:** compiled the project with `./gradlew assembleDebug` and installed the APK via `installDebug`. A compilation error in `CerebrasRecipeGenerator.kt` (missing `BuildConfig` fields) was identified and resolved.

10.3 Unit Testing Recommendations

The MVVM architecture enables isolated unit testing:

- **ViewModel tests:** inject mock repository implementations via the `AppContainer` interface and verify `StateFlow` emissions using `turbine`.
- **Repository tests:** use an in-memory Room database (`Room.inMemoryDatabaseBuilder()`) to test DAO operations without touching disk.
- **AIEngine tests:** provide a stub `CerebrasRecipeGenerator` that returns fixed responses to test the scoring and fallback logic deterministically.

10.4 UI Testing Recommendations

Jetpack Compose's testing library (`compose-ui-test`) supports semantics-based interaction:

- Use `composeTestRule.onNodeWithTag()` or `onNodeWithText()` to locate UI elements.
- Simulate user actions with `performClick()`, `performTextInput()`.
- Assert state changes with `assertIsDisplayed()`, `assertTextContains()`.

Chapter 11

Performance and Security

11.1 Performance

The NutriSmart application is designed to provide a fast and smooth user experience.

- **Asynchronous Processing:** All database and network operations run in the background using Kotlin Coroutines, ensuring the user interface remains responsive.
- **Efficient UI Updates:** The application uses reactive state management, allowing only necessary parts of the interface to update when data changes.
- **Offline Capability:** Most data is stored locally, enabling the application to function without an internet connection.

11.2 Security

Basic security measures are implemented to protect user data and application resources.

- **API Key Protection:** API keys are stored in configuration files and not directly exposed in the source code.
- **Input Validation:** User inputs such as login credentials and ingredients are validated before processing.
- **Session Management:** The user session is stored locally to maintain login state across application usage.

Chapter 12

Challenges and Solutions

12.1 BuildConfig Field Resolution

Challenge: `CerebrasRecipeGenerator.kt` referenced `BuildConfig` fields that were defined in a different Gradle module, causing a `compileDebugKotlin` compilation error.

Solution: A reflection-based workaround was implemented as a temporary measure to read the fields dynamically. The long-term solution is to consolidate `BuildConfig` field declarations in the `app` module's `build.gradle.kts` and remove the reflection dependency.

12.2 Navigation Argument Safety

Challenge: Passing navigation arguments as raw strings (e.g., `recipeId`) is prone to `NullPointerException` if the argument is missing or malformed.

Solution: `SafeNutriSmartNavGraph.kt` wraps all argument retrieval with safe-call operators and null checks. Any missing argument redirects to `ErrorFallbackScreen` with a human-readable message, completely eliminating argument-related crashes.

12.3 AI Response Parsing Robustness

Challenge: Large language models frequently include markdown code fences (````json) or conversational preamble before the JSON payload, breaking direct deserialization.

Solution: `CerebrasRecipeGenerator.kt` applies a regex-based cleaning pass to strip non-JSON content before invoking `Json.decodeFromString()`. A try/catch block wraps the entire generation pipeline and falls back to local AI results on any failure.

12.4 Immediate UI Feedback for Async Operations

Challenge: Waiting for database writes to complete before updating the UI creates a perceived lag, particularly for the favourite toggle which users may invoke repeatedly.

Solution: The ViewModel applies an *optimistic update* pattern: the in-memory `_favoriteRecipeIds` `StateFlow` is updated immediately, providing instant visual feedback, while the database write proceeds asynchronously in the background. On failure, the state is rolled back.

12.5 Deprecation Warnings

Challenge: Several API calls in the codebase use deprecated overloads (e.g., `fallbackToDestructive` and `Icons.Filled.ArrowBack`).

Solution: These are identified and documented as technical debt items. The recommended replacements are `Icons.AutoMirrored.Filled.ArrowBack` for the icon and a typed overload for the Room migration strategy.

Chapter 13

Conclusion

13.1 Summary

NutriSmart demonstrates how a well-structured Android application can deliver meaningful value in the nutrition management domain. By combining the MVVM architectural pattern, Jetpack Compose’s reactive UI model, Room’s reliable local persistence, and an extensible AI integration layer, the application addresses real user pain points: decision fatigue, food waste, budget management, and dietary adherence.

The development process produced a robust codebase characterised by:

- Clear separation of concerns across presentation, domain, and data layers.
- Defensive programming: no forced-unwrap operators, comprehensive guard checks, and centralised error handling.
- A local-first architecture that ensures offline functionality regardless of network availability.
- An AI pipeline that degrades gracefully when the remote service is unavailable.

13.2 Lessons Learned

- Investing in a robust navigation safety layer (argument validation, error fallback screens) pays dividends by eliminating an entire class of runtime crashes.
- Optimistic UI updates significantly improve perceived performance for user-initiated state changes.
- AI response parsing requires defensive cleaning due to the non-deterministic nature of language model outputs.

13.3 Future Work

Several improvements are recommended for a production release:

1. **Dependency Injection (Hilt):** replace the manual `AppContainer` with Hilt to simplify injection and enable comprehensive unit testing.
2. **Automated testing:** implement ViewModel unit tests (with Turbine), Room in-memory tests, and Compose UI tests integrated into a CI pipeline (e.g., GitHub Actions).
3. **API key security:** move the Cerebras API key behind a server-side proxy and apply R8 obfuscation.
4. **Paging:** adopt Paging 3 if the recipe catalogue grows beyond a few hundred entries.
5. **Crash reporting:** integrate Firebase Crashlytics for real-time production monitoring.
6. **Full authentication:** add email verification, password reset, and token-based session management.
7. **Deprecation resolution:** address all identified deprecated API usages before publication.

In conclusion, NutriSmart represents a solid foundation for a production-grade nutrition management application. Its modern technical stack, thoughtful architecture, and extensible AI layer position it well for the enhancements described above.